

Adaptive Layout Engine for Multi-Surface Ads

NAME: DHANUSH S J

Integrated M.Tech – Software Engineering

Email: dhanushsj2004@gmail.com

College Email: dhanush.sj2022@vitstudent.ac.in

Institution: Vellore Institute of Technology (VIT), Vellore

Resume Link: <https://drive.google.com/file/d/1pAMgHuI0MBNRbH9KCDcpcsa3u2-CmHzq/view?usp=drivesdk>

Github repository link: <https://github.com/Dhanushsj12/ai-team-intern-audit>

Portfolio link: [portfolio](#)

1. Executive Summary

The Adaptive Layout Engine for Multi-Surface Ads is a constraint-based layout resolution system designed to adapt a single advertisement specification across fundamentally different display surfaces without using surface-specific layout branches. The engine separates what the advertisement contains from how the advertisement fits a particular surface. A single advertisement specification contains semantic elements, roles, priorities, content, and layout intent. A surface profile describes the available geometry and constraints. The resolver combines both inputs and produces a valid layout containing concrete x, y, width, and height values for each placed element. The implementation avoids hardcoded conditional statements for specific surface names. Instead, layout selection is based on surface geometry, orientation, constraints, available space, element priority, and degradation rules. The project demonstrates the approach across mobile portrait, mobile landscape, broadcast lower-third, retail kiosk, a print-to-digital QR panel, an intentionally constrained banner, an unseen surface, and an equal-priority conflict scenario.

2. Project Objectives

The primary objectives are:

Single Source Declaration: Define an advertisement once and reuse the same specification across all surfaces. Realistic Surface Modeling: Model real surface constraints rather than relying only on width and height.

Geometric Diversity: Produce meaningfully different layouts instead of simple uniform scaling.

Priority Protection: Preserve higher-priority elements when space becomes constrained.

Progressive Degradation: Shrink, reposition, or drop lower-priority elements when necessary.

Overlap Prevention: Prevent element overlap through explicit 2D collision handling.

Viewport Invariant Bounds: Prevent content from being placed outside the visible surface or safe areas.

Text-Aware Layout Decisions: Model text dimensions through metric-based measurements.

DOM Rendering Integration: Render the resolved layout to real DOM/CSS environments.

Generative Surface Handling: Demonstrate generalization to an unseen surface without adding a surface-specific resolver branch.

Automated Evidence: Provide automated stress-test evidence for difficult layout cases.

3. Core Advertisement Specification

The advertisement is represented by one declarative specification.

Primary Elements:

Headline: Text element, primary role, priority 1.

Product Image: Image element, hero role, priority 1.

CTA Button: Button element, action role, priority 2.

Price Text: Text element, secondary role, priority 2.

Logo Image: Image element, branding role, priority 3.

Each element has a role and priority that influences how the resolver treats it when space becomes constrained. The same specification instance is passed to the resolver for every surface.

4. Surface Constraints

A surface profile describes the physical and interaction constraints that the layout must satisfy.

Key Surface Properties:

Physical Dimensions (Width × Height)

Safe Area Insets (Top, Right, Bottom, Left)

Minimum Tap Target size

Minimum Text Size

Viewing Distance (near, medium, far)

Touch Requirements

Different surfaces provide different constraints to the same resolver instance.

5. Required Surface Profiles

The implementation supports all required display formats:

Surface Name	Dimensions	Aspect Ratio Class	Resolution Behavior
Mobile Portrait	320 × 480	Portrait	Stacked vertical composition; secondary elements adjust position.
Mobile Landscape	480 × 320	Landscape	Split dual-column composition; full content visibility.
Broadcast Lower Third	1920 × 250	Wide Banner	Horizontal row composition; enforces far-viewing text constraints (32px + min size).
Retail Kiosk	1080 × 1080	Square	High-impact centered composition; enforces large touch targets (60px + min size).
Impossible Tight Banner	1200 × 80	Stress Test	Extreme spatial constraint; branding/secondary elements degrade gracefully.
Unseen QR Panel	600 × 900	Generative	Resolved dynamically from geometry without hardcoded configuration.

The required surfaces produce different arrangements from the same advertisement specification rather than simply scaling one common layout.

6. Constraint-Based Resolution Workflow

The layout resolution follows a systematic flow:

1. Advertisement Specification: Raw declarative input defining elements and priorities.
2. Surface Profile: Dimensions, safe area insets, and minimum sizes.

3. Geometry & Orientation Classification: Calculating overall space and aspect ratios.
4. Composition Strategy: Selecting layout flow based on geometry.
5. Priority-Ordered Resolution: Processing higher-priority elements first.
6. Constraint Checking: Validating collisions, safe areas, and minimum bounds.
7. Shrink / Reposition / Drop: Applying degradation rules where spatial conflicts occur.
8. Resolved Layout: Final relative coordinates generated for each element.
9. DOM Rendering: Painting elements to the page using calculated offset bounds.

When space becomes constrained, the resolver can keep the element at its ideal size, shrink it, reposition it into available free space, or drop it if no valid placement remains.

7. Priority and Degradation Model

Priority determines which elements receive protection during constraint resolution. The degradation process preserves the most important content first:

Priority 1: Protect primary content. Keep at ideal size whenever possible.

Priority 2: Shrink or reposition into remaining gaps where necessary.

Priority 3: Reposition into minor gaps or drop if safe space is unavailable.

The resolver never compromises a higher-priority element to make room for a lower-priority element. The resulting state of each element is explicitly recorded as either none, shrunk, repositioned, or dropped.

8. Geometry and Collision Handling

The geometry module provides spatial operations required by the resolver:

Rectangle intersection detection

Rectangle containment verification

Safe-area boundary calculations

Available-space calculations

Free-space region discovery

Before placing an element, the resolver verifies that the candidate placement remains within physical bounds, respects safe areas, avoids overlapping placed elements, satisfies minimum dimensions, and respects interaction targets.

9. Text-Aware Resolution

Text layout is not treated as a simple fixed rectangle. The project uses Canvas 2D text measurement in browser environments to estimate whether text can fit inside a candidate box.

Metric Criteria Used:

Measured text width

Font size

Line height

Available width and height

This allows the resolver to reject candidate rectangles that cannot contain rendered text. For Node-based CLI testing, a documented character-heuristic fallback is used.

10. Special Test Scenarios

Print-to-Digital QR Panel (\$600 $\times$ 900\$): Evaluated as a distinct portrait surface driven purely by geometry rather than explicit surface-name checks.

Impossible Tight Banner (\$1200 $\times$ 80\$): Deliberately constrained height forces natural degradation. The product image, CTA, price, and logo are preserved while the headline is gracefully dropped to prevent overflow.

Unseen Surface (\$2400 $\times$ 300\$): Defined only in the test suite to verify that the engine resolves unseen aspect ratios without requiring source code branch updates.

Equal-Priority Conflict (\$200 $\times$ 200\$): Two elements with equal priority competing for the same ideal position use declared specification order as the deterministic tiebreaker, repositioning the second element into remaining space.

11. Rendering & System Architecture

Division of Responsibilities:

Renderer: Responsible for pure presentation (DOM node creation, background colors, visual styling).

Resolver: Responsible for mathematical placement ($x, y, \text{width}, \text{height}$), placement state, and degradation logging.

Module Structure:

types: Core interfaces and runtime contracts.

spec: Ad specification builder and validation checks.

surfaces: Surface profile definitions.

composition-strategies: Aspect-ratio layout strategies.

resolver: Priority-ordered constraint resolution engine.

geometry: Spatial math, collision checks, and free-space search.

text-measure: Canvas 2D and fallback measurement utilities.

render-dom: Pure DOM renderer consuming resolved offsets.

stress-test: Automated CLI test runner.

12. Verification and Development Timeline

The project was completed across 5 distinct phases:

Phase 1 (Foundation): Core TypeScript types, ad specs, surface profiles, and 2D bounding box utilities.

Phase 2 (Resolver Engine): Priority placement algorithms, collision math, progressive degradation, free-space matrix search, and text measurement.

Phase 3 (Rendering & Demo): DOM layout renderer and interactive UI selector tool.

Phase 4 (Adversarial Testing): Implementing tight banner, unseen surface, and equal-priority test cases.

Phase 5 (Documentation & Verification): Architecture specs, CLI stress testing, visual screenshot generation, and live deployment.

Verification Suite Commands:

Dependency installation: ``npm install``

Development server: ``npm run dev``

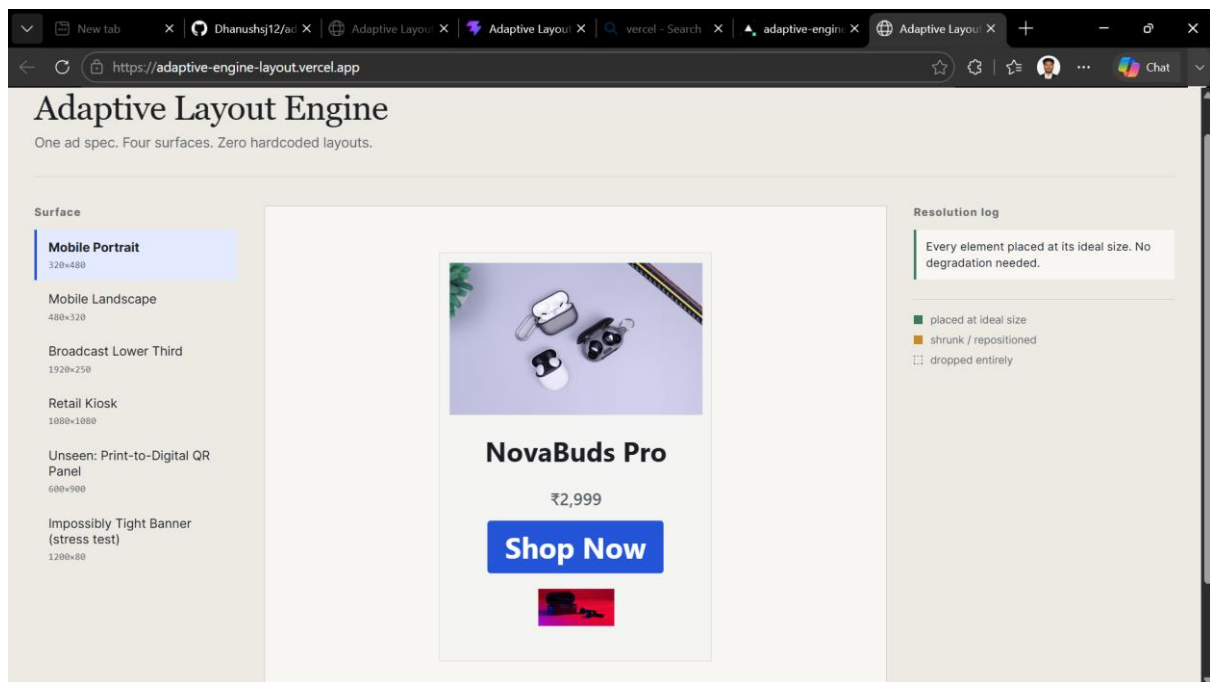
Type checking: ``npm run typecheck``

Automated stress tests: ``npm run stress-test``

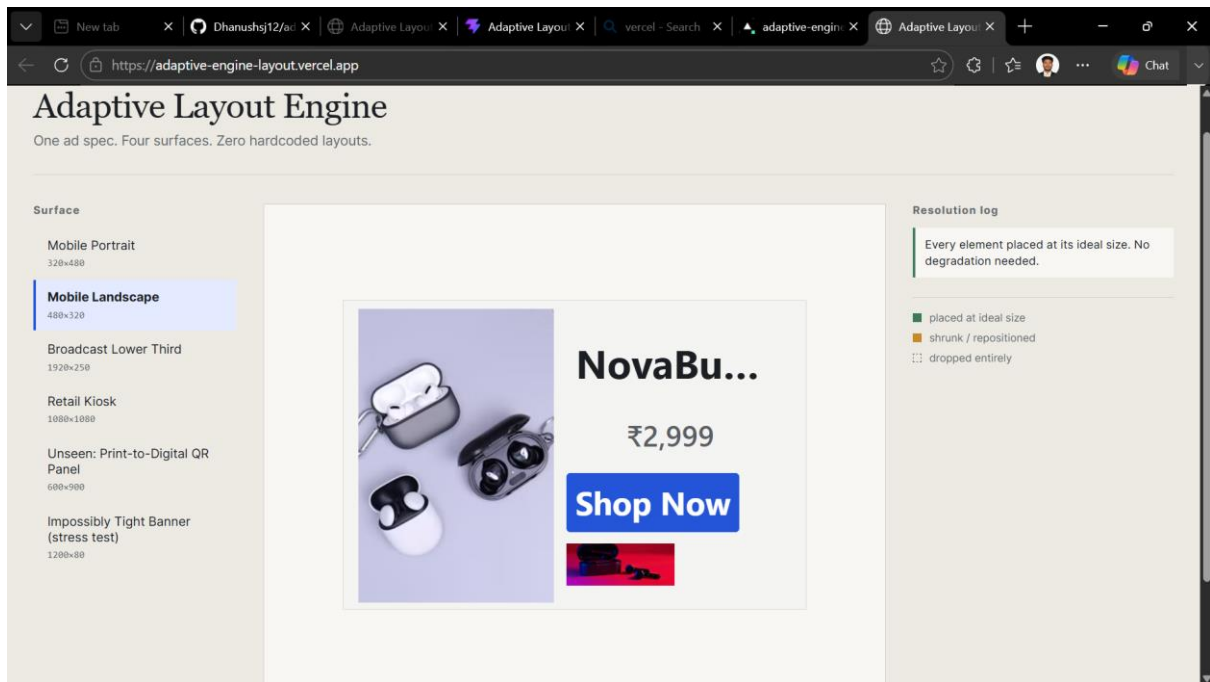
Production build: ``npm run build``

SCREENSHOTS

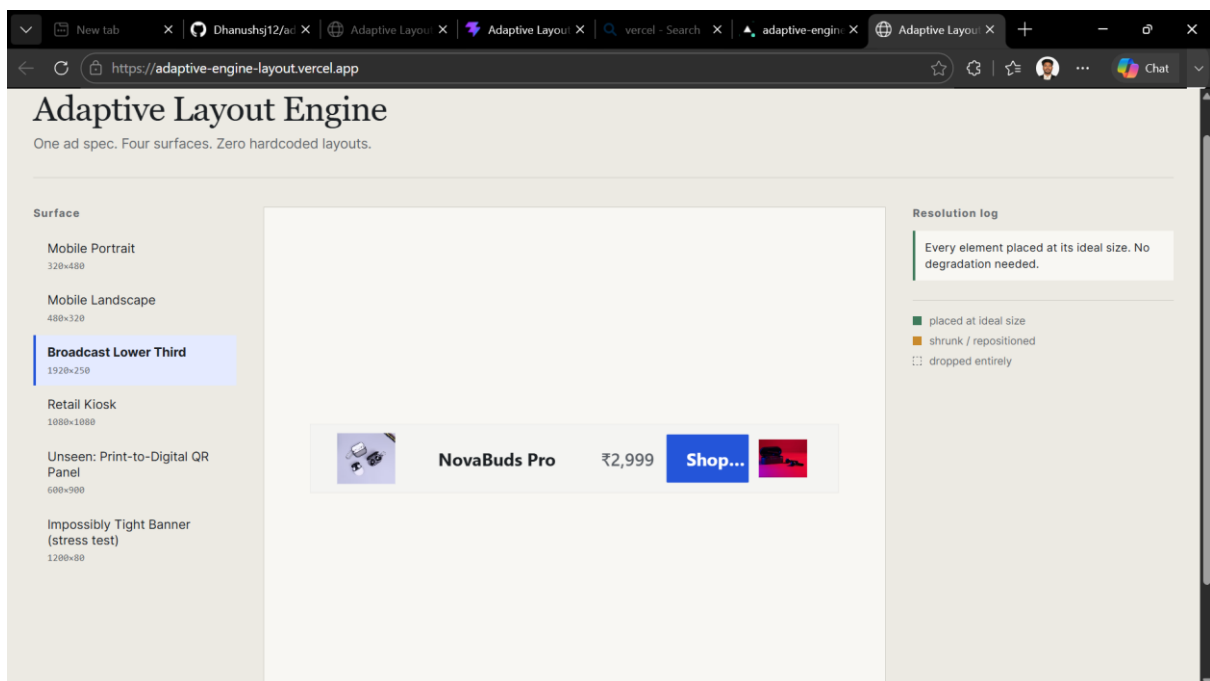
MOBILE PORTRAIT



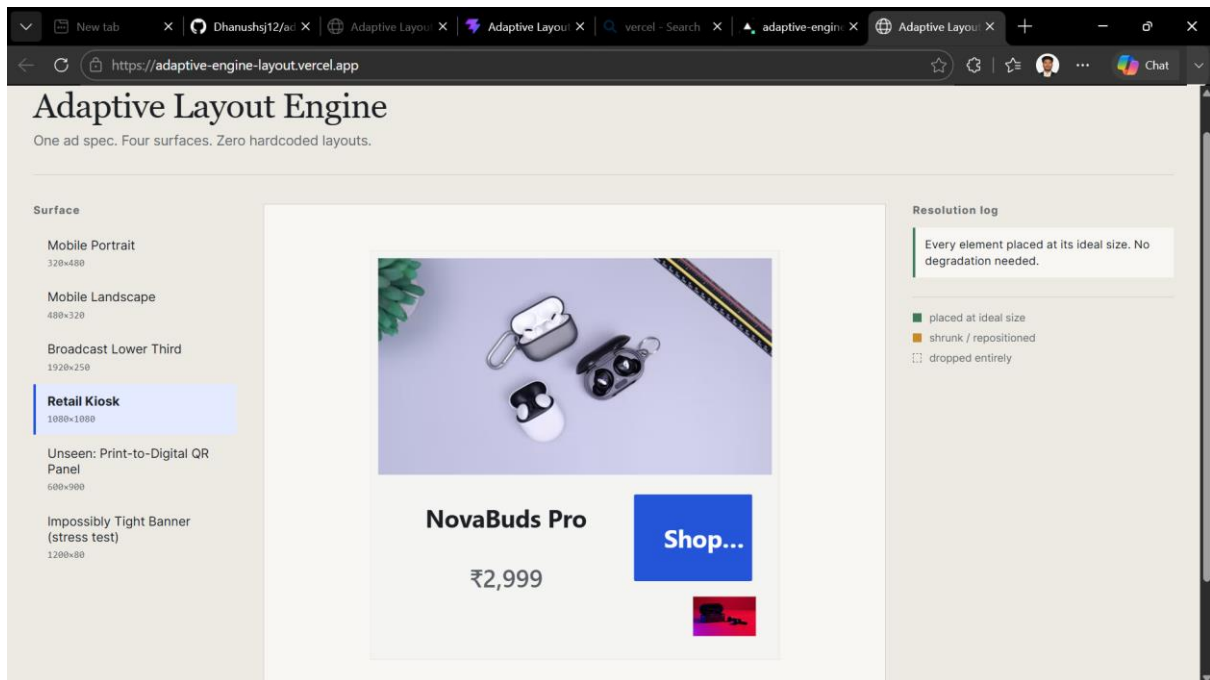
MOBILE LANDSCAPE



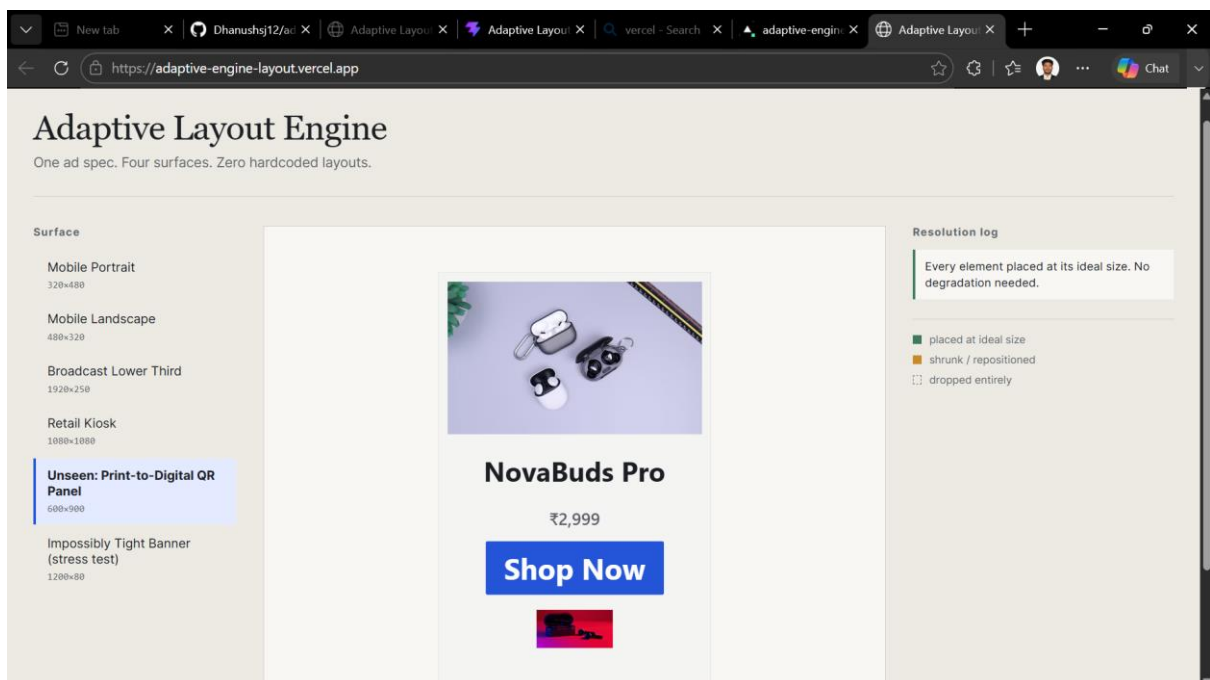
BroadCast Lower Third



Retail Kiosk



Unseen Print to Digital QR panel



Impossibly Tight Banner

